

Yep. Now that I've seen the **actual trained-fuel outputs, SHAP results, and the architecture you're following**, we can freeze the workflow and stop jumping between algorithms.

Your fuel stage is genuinely complete: the final setup uses 34 predictors, a chronological 80/20 split, and XGBRegressor achieved MAE 0.0522, RMSE 0.0880 and $R^2=0.970$. The trained pipeline is already bundled with its preprocessing and is explicitly ready to be called by the future optimization engine.

There is one **important modeling gap** we need to keep visible: the current FuelCast model has no true cargo/load or draft variable; the report explicitly flags that. That means we should **not pretend XGB currently understands cargo loading effects**. We can still use cargo in the optimizer, but later we need to decide whether we enrich/retrain the fuel model or use an explicitly defined approximation for load effects.

The final procedure

PHASE 1 — Data layer

You have already done this.

```
Ship / Engine / AIS
Weather / Ocean
Fleet
Port
Cargo / Demand
Fuel
Cost
GHG
Regulatory
```

Your different datasets should remain **separate by purpose**.

Do not merge everything into one giant CSV.

PHASE 2 — Data processing

Already completed for your current datasets.

For optimization, the next job is not "train them."

Instead, create **optimization-ready parameter tables**.

For example:

```
Fleet parameters
Port parameters
Cargo-demand parameters
Fuel parameters
GHG parameters
Cost parameters
```

Think:

raw datasets → parameters used by equations
 $\text{raw datasets} \rightarrow \text{parameters used by equations}$

PHASE 3 — Mathematical + physics layer

This is the next thing I would formally finish.

Define the relationships that connect your decisions.

Voyage time

$$T_{ij} = D_{ij} / V_i \quad T_{ij} = \frac{D_{ij}}{V_i}$$

Load ratio

$$LR_i = \text{Cargo}_i / \text{DWT}_i \quad LR_i = \frac{\text{Cargo}_i}{\text{DWT}_i}$$

Fuel

For now:

$$F_i = XGB(x_i) \quad F_i = XGB(x_i)$$

where x_i contains the variables the actual model supports.

The existing fuel report confirms the model currently uses speed, weather/sea-state variables and engineered fuel-mix features, while load/draft are unavailable.

Cost

$$C_i = F_i P_{\text{fuel},i} + C_{\text{port},i} + C_{\text{operation},i} + C_{\text{shore},i} \quad C_i = F_i P_{\text{fuel},i} + C_{\text{port},i} + C_{\text{operation},i} + C_{\text{shore},i}$$

GHG

$$G_i = F_i(EFWTT_i + EFTTW_i) \quad G_i = F_i(EF_{WTT,i} + EF_{TTW,i})$$

Then totals:

$$F_{total} = \sum_i F_i \quad F_{total} = \sum_i F_i \quad C_{total} = \sum_i C_i \quad C_{total} = \sum_i C_i \quad G_{total} = \sum_i G_i \quad G_{total} = \sum_i G_i$$

This gives your optimization objectives:

$$\min[F_{total}, C_{total}, G_{total}] \quad \boxed{\min [F_{total}, C_{total}, G_{total}]}$$

PHASE 4 — Define the decisions

This is where **fleet optimization** actually begins.

Your optimizer can choose:

Vessel

$$x_{ij} \in \{0, 1\} \quad x_{ij} \in \{0, 1\}$$

Vessel i serves demand j .

Fuel

$$y_{im} \in \{0, 1\} \quad y_{im} \in \{0, 1\}$$

Vessel i uses fuel m .

Route

$$r_{ik} \in \{0, 1\} \quad r_{ik} \in \{0, 1\}$$

Speed

Either continuous:

$$V_i \in [V_{min}, V_{max}] \quad V_i \in [V_i^{\min}, V_i^{\max}]$$

or discrete speed choices:

$$z_{iv} \in \{0, 1\} \quad z_{iv} \in \{0, 1\}$$

Cargo allocation

$q_{ij} \geq 0$

Shore power

$s_i \in \{0, 1\}$

Now one candidate is something like:

```
Vessel B
Mumbai → Chennai
10,000 t
13 knots
LNG
Shore power = Yes
```

PHASE 5 — Candidate generation

Now your fleet/port/cargo data becomes useful.

Generate possible combinations:

```
Vessel
+
Demand
+
Route
+
Speed
+
Fuel
+
Shore power
```

But **don't blindly generate every combination** if the number becomes huge.

Use obvious domain rules first.

Example:

```
LNG unavailable at port
→ don't generate LNG candidate
```

```
Vessel capacity < cargo  
→ don't generate candidate
```

```
Port draft too shallow  
→ don't generate candidate
```

This is your **candidate generator**.

PHASE 6 — Hard feasibility filtering

This is where I would keep the system deterministic.

Check:

```
Cargo ≤ Capacity Cargo \le Capacity Draft ≤ PortDepth Draft \le PortDepth Vmin ≤ V ≤ Vmax V_{min} \le V \le V_{max} T ≤ Deadline T \le Deadline FuelUsed ≤ FuelAvailable FuelUsed \le FuelAvailable PortFlow ≤ PortCapacity PortFlow \le PortCapacity
```

and vessel/fuel compatibility.

Your architecture image already follows this logic: candidate generation → classical constraint checks → feasible candidate set.

Result:

```
Millions of theoretical combinations  
↓  
Hard constraints  
↓  
Feasible candidate set
```

PHASE 7 — Call the trained XGB pipeline

Now, **finally**, the fuel model enters the optimization.

For every feasible candidate:

```
candidate  
↓  
fuel_xgb_pipeline.joblib
```

↓
predicted_fuel

The saved artifact is specifically designed for this: raw scenario → preprocessing → XGBRegressor → predicted fuel.

So your optimizer will eventually call something conceptually like:

```
predicted_fuel = fuel_pipeline.predict(candidate)
```

BUT important

Because the current model doesn't contain cargo/load/draft as inputs, don't pass fake load variables to it.

This is the issue we need to resolve before final optimization runs.

PHASE 8 — Calculate objectives

Once fuel is predicted:

```
Candidate
  ↓
Predicted Fuel
  ├──> Fuel objective
  ├──> Fuel cost
  └──> GHG
```

So each feasible candidate becomes:

$x \rightarrow (F(x), C(x), G(x))$

Example:

Candidate	Fuel	Cost	GHG
A	420	120k	850
B	450	105k	720
C	390	140k	600

Now we have an actual multi-objective problem.

PHASE 9 — CQM formulation

Now the architecture's CQM block makes sense.

Build the **full constrained mathematical model**:

```
Decision variables
+
Objective(s)
+
Constraints
    ↓
    CQM
```

For example:

$$\sum_{i,j} x_{ij} = 1$$

one vessel serves the demand,

$$q_{ij} \leq \text{Cap}_i x_{ij} \quad \forall i, j$$

capacity,

$$T_{ij} \leq \text{Deadline}_j x_{ij} \quad \forall i, j$$

deadline.

Important:

CQM is **not a separate optimizer that optimizes one constraint at a time**.

It is your **constraint-aware mathematical representation**.

PHASE 10 — Convert the binary part to QUBO

Now comes your quantum formulation.

For binary variables:

$$x_i \in \{0,1\} \quad x_i \in \{0,1\}$$

create:

$$Q(x) = x^T Q x \quad \boxed{Q(x) = x^T Q x}$$

Constraints become penalty terms.

Example:

$$x_1 + x_2 + x_3 = 1 \quad x_1 + x_2 + x_3 = 1$$

becomes:

$$\lambda (x_1 + x_2 + x_3 - 1)^2 \quad \lambda (x_1 + x_2 + x_3 - 1)^2$$

inside the QUBO objective.

So:

Full constrained problem

↓

CQM

↓

Binary encoding

↓

QUBO

This is where the quantum/QI branch gets mathematically connected.

PHASE 11 — Classical baseline: NSGA-II

This is your baseline.

NSGA-II receives:

$$[F(x), C(x), G(x)] \quad [F(x), C(x), G(x)]$$

and searches for non-dominated solutions.

Its loop is:

Initial population

↓

Evaluate Fuel / Cost / GHG


```

      ↓
Pareto ranking
      ↓
Selection
      ↓
Crossover
      ↓
Mutation
      ↓
Constraint handling
      ↓
Next generation
      ↓
Repeat

```

Output:

Pareto front $\boxed{\text{Pareto front}}$

This is your **classical benchmark**.

PHASE 12 — Quantum-inspired side: MO-QIGA

Now use the **same problem**:

```

same candidate representation
same objectives
same constraints
same fuel predictions

```

but change the search representation/update mechanism.

Your quantum-inspired representation can use Q-bits:

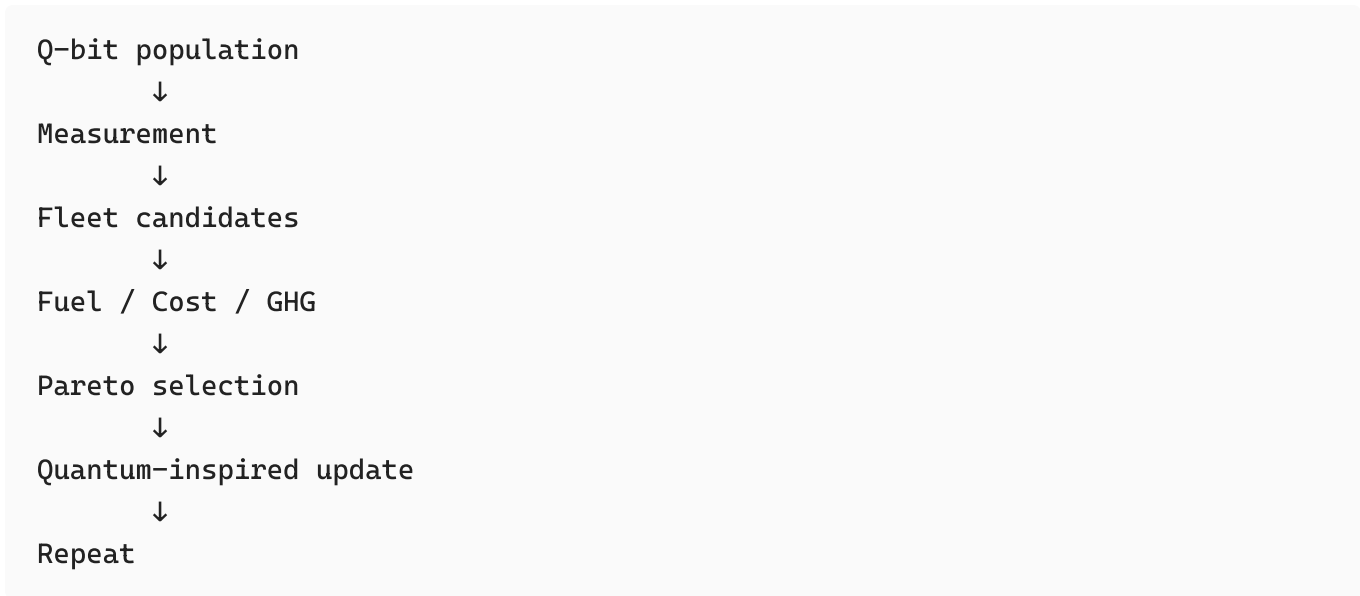
$$q_i = (\alpha_i, \beta_i) \quad q_i = (\alpha_i, \beta_i)$$

where:

$$|\alpha_i|^2 + |\beta_i|^2 = 1 \quad |\alpha_i|^2 + |\beta_i|^2 = 1$$

Measurement produces binary candidate decisions.

Then your MO-QIGA performs:



Output:

Quantum-inspired Pareto front

PHASE 13 — Compare NSGA-II vs MO-QIGA

This is where your actual research contribution becomes measurable.

Both receive:

- SAME
 - ✓ data
 - ✓ XGB fuel model
 - ✓ candidate space
 - ✓ objectives
 - ✓ constraints

Only the optimization/search mechanism differs.

Compare:

Solution quality

HypervolumeHypervolume IGDIGD

Computational performance

RuntimeRuntime

Search behaviour

ConvergenceConvergence

Feasibility

Constraint violation rateConstraint\ violation\ rate

Diversity

How well the Pareto solutions cover different fuel/cost/GHG trade-offs.

PHASE 14 — MILP as validation

Don't put MILP in the main pipeline as if it must run after NSGA-II.

Use it separately:

```
Small problem instance
      ↓
MILP
      ↓
Exact/near-exact reference
```

Then compare NSGA-II and MO-QIGA against it.

This tells you:

"How close are our evolutionary/QI solutions to an exact reference on problems small enough to solve exactly?"

That's a **validation benchmark**, not a required stage for every large instance.

PHASE 15 — Robustness testing

Once the optimizer works, perturb:

```
Weather
Cargo demand
Fuel price
```

Fuel availability
Port conditions

Then rerun.

For example:

Demand'=1.1DemandDemand' = 1.1Demand Price'=1.2PricePrice' = 1.2Price

etc.

Now you can ask:

| Does the recommended fleet plan remain useful under changing conditions?

PHASE 16 — Final output

Your system finally gives:

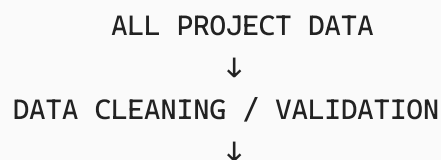
Recommended fleet plan

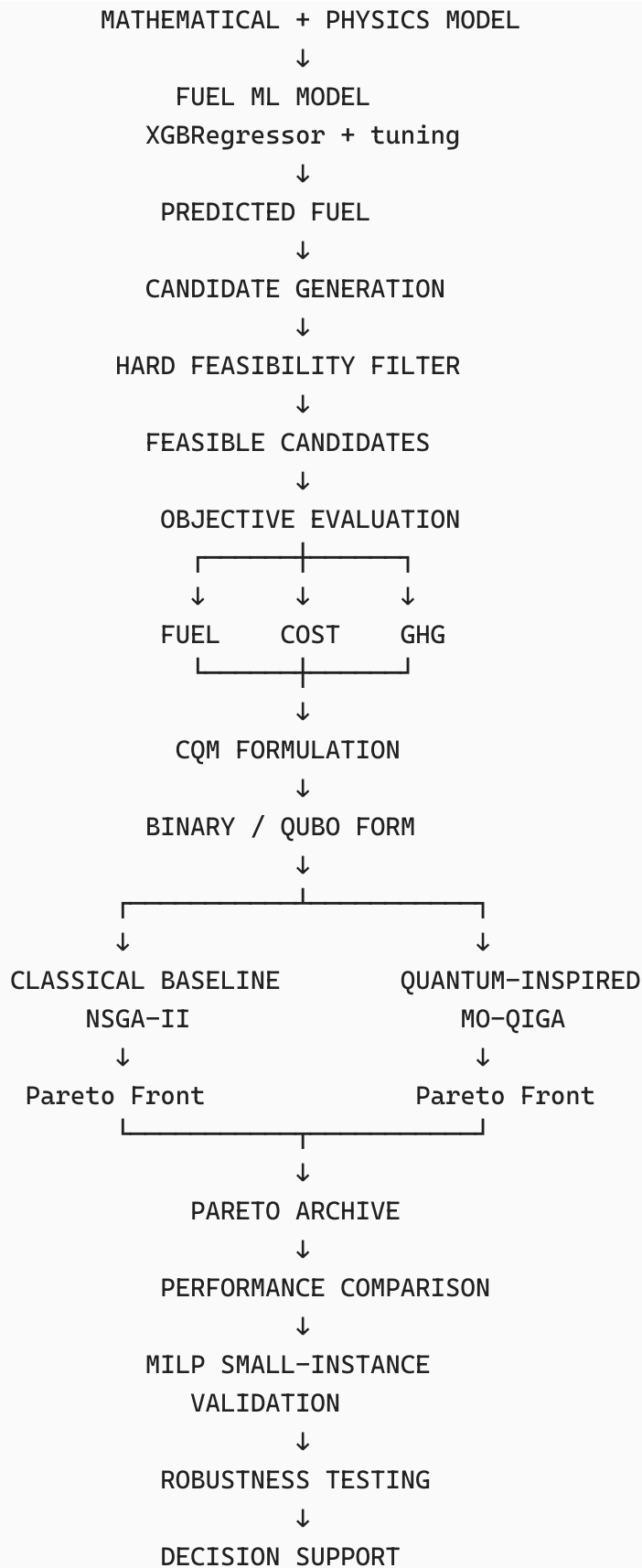
- |— Vessel
- |— Route
- |— Speed
- |— Fuel
- |— Cargo allocation
- |— Shore power
- |— Fuel consumption
- |— Cost
- |— GHG
- |— Constraint status

And, importantly, **several Pareto options**, not one forced answer.

Your exact final architecture

I would now freeze it as:





And where NLP fits

Your second architecture has an **NLP/LLM constraint parser**. I'd keep that as an **optional user-interface layer**, not as part of the core optimization engine.

For example:

“Deliver 5,000 tonnes to Chennai within 48 hours using LNG-compatible vessels.”

NLP converts that into structured constraints:

$Cargo \geq 5000$ $Cargo \leq 5000$ $T \leq 48$ $T \leq 48$ $FuelType = LNG$ $FuelType = LNG$

Then those constraints enter the **same CQM/optimization formulation**.

So:

```
Natural language
  ↓
NLP parser
  ↓
Structured constraints
  ↓
CQM
```

That is a really nice front-end feature, but **don't let it complicate your core optimizer**.

● One issue we should address BEFORE optimization coding

Your current fuel model's own report says:

no explicit timestamp, no true draft/loading condition, and no load variable.

So if your optimizer chooses:

```
Cargo = 10,000 t
```

but the XGB model doesn't receive cargo/load, then changing:

```
Cargo = 5,000
```

to

```
Cargo = 15,000
```

won't change the XGB fuel prediction.

That is not okay if we're claiming that cargo affects fuel.

So **before implementing the optimizer**, we need to decide how we're going to handle this missing relationship.

This is the one technical issue I would resolve next.

Your immediate next steps, in order

- 1. Lock the mathematical formulation.**
- 2. Build the optimization parameter tables from your cleaned fleet/port/cargo/fuel/GHG data.**
- 3. Define the exact candidate representation.**
- 4. Implement hard feasibility filtering.**
- 5. Connect `fuel_xgb_pipeline.joblib`.**
- 6. Implement objective calculation.**
- 7. Implement CQM/QUBO representation.**
- 8. Implement NSGA-II baseline.**
- 9. Implement MO-QIGA.**
- 10. Compare them + MILP on small instances.**
- 11. Robustness testing.**

That is the clean path from where you are **right now** to a working optimization system.